Group Members:
- Hernaez, Raeka
- Liwanag, Ram
- Serrano, Paul
- Tiu, Kyle

Project Title:

SIMD-BASED AVX2 MATRIX MULTIPLICATION

Abstract

Matrix multiplication is a mathematical operation used in various computing applications. This project aims to develop a program that performs matrix multiplication using AVX2 assembly to demonstrate how multiple data values can be processed at the same time. The program will generate two matrices with compatible dimensions and calculate their product using two approaches, a sequential implementation that processes the calculations one at a time and a parallel implementation using AVX2 assembly, which allows multiple values to be processed in a single instruction. The results of both implementations will be checked for correctness, and their execution times will be measured and compared across different matrix sizes. Through this comparison, the project aims to determine how using SIMD affects the execution time of matrix multiplication as the input size increases.

Proposed Project Description

a. What are the **inputs** to your project?

The program will take in two matrices, A and B, that are populated with single-precision floating-point integers. The user will have to input four different dimension inputs: X rows and Y columns for matrix A, and Y rows and Z columns for matrix B. As it is imperative that the number of columns for matrix A and the number of rows for matrix B are equal, error checking will be implemented to ensure that this is enforced. These values will be used to construct the matrixes, which will then be filled with randomly generated single-precision floating-point integers.

Example input:
ENTER MATRIX A ROWS: 2
ENTER MATRIX A COLUMNS: 3
ENTER MATRIX A ROWS: 3
ENTER MATRIX A COLUMNS: 4

b. Proposed Process

i. **What processes will be parallelized** using SIMT (GPU/CUDA) or SIMD-based AVX/AVX2/AVX-512 assembly?

The primary processes that will be parallelized is the multiplication of the matrix elements and the subsequent addition of their products using SIMD-based AVX2 assembly. Because the usual implementation of matrix multiplication in programming will have the computation be looped for each element one by one, execution times may take longer depending on the number of elements to be calculated. As such, parallelizing these processes will definitely increase efficiency by allowing multiple data elements to be processed using less instructions

ii. What existing implementations have been done with the proposed process (i.e., **related works**)? **How would it differ** from your proposed implementation if it has been parallelized?

One related work is Performance Evaluation of Matrix-Matrix Multiplications Using Intel's Advanced Vector Extensions (AVX) by Hassan, Hemeida, and Mahmoud (2016). They studied and tested different programming methods, including inline assembly and intrinsic functions, using C++ compilers. Their results showed that intrinsic functions performed better than inline assembly in their tests, and the intel compiler generally performed better than the Microsoft compiler. This study is related to our project because it also explores the use of vector instructions for matrix multiplication.

Another is the Basic Linear Algebra Subprograms (BLAS) which is a library of routines for performing linear algebra operations, including matrix-matrix multiplication. It is relevant to this project because it provides an established implementation of the same computational process.

Unlike the related study, which focused on comparing programming methods and compilers for AVX-based matrix multiplication, our project will focus on applying AVX2 assembly to matrix multiplication and examining how it affects execution time compared with a sequential implementation. While BLAS provides general-purpose routines for matrix operations, our project focuses on implementing matrix multiplication AVX2 SIMD assembly and comparing its execution time with a sequential implementation. Our objective is to demonstrate what we have learned about SIMD by implementing matrix multiplication using AVX2 assembly and demonstrating how data level parallelism can be used to perform the computation, rather than to outperform existing libraries or implementations

iii. Discuss plans on **how to parallelize** your proposed process

To parallelize the processes, the group plans to use packed symmetric operations such as VADDPS and VMULPS to speed up the computational portion of the program. A sequential implementation of matrix multiplication will be made in the main caller program using typical nested loops and basic addition and multiplication operations. This program will serve as a benchmark to test the parallelized program against, as both implementations will be measured for their execution time and how much faster one is compared to the other. Comparisons and observations will be made on how each implementation deals with increasing dimension sizes and in what areas one program trumps the other.

iv. What other processes are in the project but will not be parallelized

Supporting operations like matrix element generation, memory allocation, and execution time measurement will be included in the project but implemented normally. Additionally, a sequential implementation of the matrix multiplication process will be used as a basis for comparison. As this project is focused on the acceleration of numerical calculations, the group felt that these processes do not necessarily need to be parallelized in the context of the group's project.

c. What are the **outputs** of your project?

The program will produce the matrix results after multiplying Matrix A and Matrix B, along with the performance analysis when comparing the sequential implementation and the AVX2 SIMD assembly implementation. The output matrix will contain single-precision floating-point values calculated from the two input matrices. The program will also display the execution time of both implementations to quantify the performance improvement from SIMD parallelization. It will calculate the speedup ratio by dividing the sequential execution time by the AVX2 execution time. These measurements will be gathered across different matrix sizes to be able to observe how the size of the input matrices affects the performance of the program. Likewise, the program will verify whether the results produced by both implementations are equal within an acceptable floating-point tolerance, making sure that the AVX2 implementation will produce correct results while improving computational performance.

Example Output:

AVX2 MATRIX MULTIPLICATION

Matrix A Dimensions: 2 x 3
GENERATED MATRIX DISPLAYED HERE

Matrix B Dimensions: 3 x 4
GENERATED MATRIX DISPLAYED HERE

Resulting Matrix C (A x B):
RESULTING MATRIX DISPLAYED HERE

PERFORMANCE COMPARISON
Sequential Execution Time: 0.020 ms
AVX2 Execution Time: 0.015ms
Speedup: 1.33x

Result Verification: PASSED *(Show passed or failed to check for errors on the output calculations and tolerance.)*

The final project deliverables are as follows:

1. Source codes for the sequential and AVX2 assembly implementations of matrix multiplication
2. Comparison between the two implementations in terms of execution time and speedup across several dimension sizes
3. Discussions on effects of matrix size on time efficiency and the effectiveness of SIMD parallelization in improving computational performance
4. Documentation of the program's functionality, implementation process, and group’s thinking process

Proposed **target implementation platform** of the proposed project

This project will be implemented using C with AVX2 assembly to develop both the sequential and SIMD-based matrix multiplication implementations. The sequential implementation will serve as the baseline for comparison with the AVX2 implementation, which will use SIMD instructions to perform operations on multiple single-precision floating-point values simultaneously

The program will be developed and compiled using Visual Studio 2026 where it will also be run on. The hardware must include a processor that supports the AVX2 instruction set to ensure the implementation can execute correctly. The program will accept compatible matrices as inputs, calculate their product using both implementations, and compare their execution times and numerical values to evaluate the application of data level parallelism

Mid-project milestones of the proposed project

Before the Week 10 demo, we aim to complete the sequential and AVX2 matrix multiplication implementations, including matrix generation and input validation. We will also implement the result verification and execution time comparisons that will be needed to calculate the speedup. The demo should showcase both implementations, verify results, and initial performance measurements using different matrix sizes.

References:

[1] Hassan, S. A., Hemeida, A. M., & Mahmoud, A. A. (2016). Performance evaluation of matrix-matrix multiplications using Intel's advanced vector extensions (AVX). Microprocessors and Microsystems, 47(B), 369–374. https://www.sciencedirect.com/science/article/abs/pii/S0141933116302502

[2] Netlib. (n.d.). BLAS (Basic Linear Algebra Subprograms). https://www.netlib.org/blas/